

F1系统接口清单【外发版本】

1. 关节电机限位说明

通过ros2 topic /motion_ctl/joint_ctl，传输控制相应电机的角度，当传入的控制角度超出限位时，电机不会旋转，在ros2 motor_ctl节点的日志文件 /home/user/workspace/logs2/motor_ctl.log中，可以查看到对应调用的错误日志。

```
1 [motor_ctl]: Joint limit exceeded! arm_r_j4
```

实际使用时，避免传入超出限位的角度值！！

详细信息参考：

/home/user/workspace/f1/install/motor_ctl/share/motor_ctl/launch/motor_ctl.launch.xml

手臂：

1	轴位（功能）	运动范围
2	J1 轴（旋转）	-178° ~ +178°
3	J2 轴（下臂）	-120° ~ +120°
4	J3 轴（上臂）	-178° ~ +178°
5	J4 轴（手腕旋转）	-145° ~ +60°
6	J5 轴（手腕摆动）	-178° ~ +178°
7	J6 轴（手腕回转）	-60° ~ +60°
8	J7 轴（手腕回转）	-90° ~ +90°

头部及躯干部：

1	序号 电机名称 限位角度及方向说明
2	0 head_j1:-40° ~ +40°，yaw，摇头，正向为左摇头
3	1 head_j2:-25° ~ +25°，pitch，抬头，正向为上抬头
4	2 head_j3:-15° ~ +15°，roll，歪头，正向为左歪头
5	3 body_j1:-130° ~ +130°，腰部yaw，正向为向左
6	4 body_j2:-10° ~ +10°，腰部roll，正向为向左
7	5 body_j3:-70° ~ 0°，腰部pitch，只能俯身（方向为负），不可后仰，只能为负值，绝对值越大越前倾
8	6 body_j4:-55° ~ 0°，屈膝，绝对值越大姿态越高，只能为负值

9 7 body_j5: -30° ~ 0°, 脚踝, 绝对值越大姿态越高, 只能为负值

2. 接口使用与节点启动概述

需要user用户, 执行如下脚本 /home/user/workspace/start_all.sh ,
启动成功后, 进行后续的接口调用。

文档位置: /home/user/workspace/f1-system-readme

使用说明: 通过文档, 可执行脚本, 将电机及手臂调至指定位置, 如何启动系统的ros2节点。

启动ros2节点之后, 可以使用本文档第三部分的Topic, 以及使用本文档第四部分的Service。

注意:

- a. 控制头部及躯干部分的电机, 发送ros2控制topic, 不可以传入较大变化的角度控制参数!
- b. 控制手臂电机, 通过ros2发送控制msg时, 必须传入整条手臂的7个电机参数。即使只需要调整一个关节, 其余6个关节可以传入当前的电机角度。

```
1 #####
2 执行/home/user/workspace/start_all.sh
3 启动系统ros节点: 电机控制, 夹爪, 头部相机(额头), 两个腕部相机
4
5 #####
6 start_ec_master 启动ec主站
7 已配置开机自启动
8
9 重启主站方式: sudo systemctl restart ec-master.service
10
11 #####
12 调节腰部至工作高度, 结束位置ends可配置(需查看限位说明)
13 脚本在~/workspace/f1/test 目录
14 starts的三个起始位置需要读取 /hal/joint_states中
15 - body_j3
16 - body_j4
17 - body_j5
18 这三个关节的数据
19 !! 此处一定不能出错!!
20 执行以下脚本, 进行调节
21 python3 ros2_joint_control.py --motors 5,6,7 --starts="-4,-20,-12" --
ends="-2,-6,-5" --duration 10
22
23 #####
```

```

24  调节头部至零位
25  脚本在~/workspace/f1/test 目录
26  starts的三个起始位置需要读取 /hal/joint_states中
27  - head_j1
28  - head_j2
29  - head_j3
30  这三个关节的数据
31  !! 此处一定不能出错!!
32  执行以下脚本，进行调节
33  python3 ros2_joint_control.py --motors 0,1,2 --starts="-36,-3.3,-10.5" --
    ends="-0.1,-0.1,-0.1" --duration 10
34
35  #####
36  调节双臂至工作位
37  ros2 topic pub /motion_ctl/joint_ctl sensor_msgs/msg/JointState "{ header:
    {stamp: {sec: 0, nanosec: 0}, frame_id: 'base_link'}, name: [
    'arm_l_j1','arm_l_j2','arm_l_j3','arm_l_j4','arm_l_j5','arm_l_j6','arm_l_j7',
    'arm_r_j1','arm_r_j2','arm_r_j3','arm_r_j4','arm_r_j5','arm_r_j6','arm_r_j7'
    ], position: [90.0, -90.0, -90.0, -90.0, 0.0, 0.0, 0.0, -90.0, -90.0, 90.0,
    -90.0, 0.0, 0.0, 0.0], velocity: [], effort: []}" --once
38

```

3. Ros2 Topic列表

3.1.1 关节控制（手臂）

注意：

- ① 由于天机sdk接口限制，单条手臂控制时，7个关节的角度值，都需要传入！！
- ② 超出控制的限时时，ros2日志会出现报错信息，避免传入超限的角度。参考本文第一部分。

```

1  [motor_ctl_node-1] [ERROR] [1779866506.421479027] [motor_ctl]: Joint
    [arm_r_j4] position -145.00° exceeds limit [-140.00, 55.00]
2  [motor_ctl_node-1] [ERROR] [1779866506.421540630] [motor_ctl]: Joint limit
    exceeded! arm_r_j4

```

Topic	/motion_ctl/joint_ctl
消息类型	sensor_msgs::msg::JointState

接入方式	订阅（具身→系统）
接口说明	控制关节运动
接口参数	header：标准消息头，包含时间戳（stamp）、坐标系（frame_id）等元信息 string[] name：关节名称列表 double64[] position：指定每个关节的目标位置，控制机器人关节的旋转角度 double64[] velocity： double64[] effort：

双臂运动示例

```
1 ros2 topic pub /motion_ctl/joint_ctl sensor_msgs/msg/JointState "{header:
{stamp: {sec: 0, nanosec: 0}, frame_id: 'base_link'}, name: ['arm_l_j1',
'arm_l_j2', 'arm_l_j3', 'arm_l_j4', 'arm_l_j5', 'arm_l_j6', 'arm_l_j7',
'arm_r_j1', 'arm_r_j2', 'arm_r_j3', 'arm_r_j4', 'arm_r_j5', 'arm_r_j6',
'arm_r_j7'], position: [21.8, -41.0, -4.74, -63.67, 10.15, 14.72, 7.68,
-21.8, -41.0, 4.75, -63.67, -10.15, 14.72, -7.68], velocity: [], effort: []}"
--once
```

双臂归位示例

```
1 ros2 topic pub /motion_ctl/joint_ctl sensor_msgs/msg/JointState "{header:
{stamp: {sec: 0, nanosec: 0}, frame_id: 'base_link'}, name: ['arm_l_j1',
'arm_l_j2', 'arm_l_j3', 'arm_l_j4', 'arm_l_j5', 'arm_l_j6', 'arm_l_j7',
'arm_r_j1', 'arm_r_j2', 'arm_r_j3', 'arm_r_j4', 'arm_r_j5', 'arm_r_j6',
'arm_r_j7'], position: [90.0, -90.0, -90.0, 0.0, 0.0, 0.0, 0.0, -90.0, -90.0,
90.0, 0.0, 0.0, 0.0, 0.0], velocity: [], effort: []}" --once
```

双臂工作位

```
1 ros2 topic pub /motion_ctl/joint_ctl sensor_msgs/msg/JointState "{ header:
{stamp: {sec: 0, nanosec: 0}, frame_id: 'base_link'}, name: [
'arm_l_j1','arm_l_j2','arm_l_j3','arm_l_j4','arm_l_j5','arm_l_j6','arm_l_j7',

'arm_r_j1','arm_r_j2','arm_r_j3','arm_r_j4','arm_r_j5','arm_r_j6','arm_r_j7'
], position: [90.0, -90.0, -90.0, -90.0, 0.0, 0.0, 0.0, -90.0, -90.0, 90.0,
-90.0, 0.0, 0.0, 0.0], velocity: [], effort: []}" --once
```

3.1.2 手臂末端位姿控制（未设置工具）

左臂

Topic	/motion_ctl/left_arm/tcp_pos_ctl
消息类型	sensor_msgs::msg::JointState
接入方式	订阅（具身→系统）
接口说明	通过末端位姿，控制关节运动
接口参数	header：标准消息头，包含时间戳（stamp）、坐标系（frame_id）等元信息 string[] name：size为6数组，名称分别是 'x', 'y', 'z', 'rx', 'ry', 'rz', 'use_nsp', 'nsp_type', 'nsp_p1', 'nsp_p2', 'nsp_p3', 'nsp_angle' double64[] position：分别对应"x" "y" "z" "rx" "ry" "rz"的参数值，xyz单位为mm，rxryrz单位为度， use_nsp的值为0或1，表示关闭或开启零空间优化； nsp_type的值为0或1，零空间类型：0-近参考（NEAR_REF），1-近方向（NEAR_DIR） nsp_params：零空间方向参数，通过计算得出，详见3.1.3 nsp_angle： double64[] velocity： double64[] effort：

Code block

```
1 ros2 topic pub /motion_ctl/left_arm/tcp_pos_ctl sensor_msgs/msg/JointState "{
2   header: {stamp: {sec: 0, nanosec: 0}, frame_id: 'base_link'},
3   name:
4   ['x','y','z','rx','ry','rz','use_nsp','nsp_type','nsp_p1','nsp_p2','nsp_p3','ns
5   p_angle'],
6   position: [303.0, 290.6, 255.2, -95.0, 11.0, -86.0, 1, 1, 0.6765, -0.7364,
7   0.0, 15.0],
8   velocity: [],
9   effort: []
10 }" --once
11 publisher: beginning loop
12 publishing #1:
13 sensor_msgs.msg.JointState(header=std_msgs.msg.Header(stamp=builtin_interfaces.
14 msg.Time(sec=0, nanosec=0), frame_id='base_link'), name=['x', 'y', 'z', 'rx',
15 'ry', 'rz', 'use_nsp', 'nsp_type', 'nsp_p1', 'nsp_p2', 'nsp_p3', 'nsp_angle'],
```

position=[303.0, 290.6, 255.2, -95.0, 11.0, -86.0, 1.0, 1.0, 0.6765, -0.7364, 0.0, 15.0], velocity=[], effort=[])

右臂

Topic	/motion_ctl/right_arm/tcp_pos_ctl
消息类型	sensor_msgs::msg::JointState
接入方式	订阅（具身→系统）
接口说明	控制关节运动
接口参数	header：标准消息头，包含时间戳（stamp）、坐标系（frame_id）等元信息 string[] name：size为6数组，名称分别是 'x', 'y', 'z', 'rx', 'ry', 'rz', 'use_nsp', 'nsp_type', 'nsp_p1', 'nsp_p2', 'nsp_p3', 'nsp_angle' double64[] position：分别对应"x" "y" "z" "rx" "ry" "rz"的参数值，xyz单位为mm，rxryrz单位为度， use_nsp的值为0或1，表示关闭或开启零空间优化； nsp_type的值为0或1，零空间类型：0-近参考（NEAR_REF），1-近方向（NEAR_DIR） nsp_params：零空间方向参数，通过计算得出，详见3.1.3 nsp_angle： double64[] velocity： double64[] effort：

Code block

```
1  ros2 topic pub /motion_ctl/right_arm/tcp_pos_ctl sensor_msgs/msg/JointState "{
2    header: {stamp: {sec: 0, nanosec: 0}, frame_id: 'base_link'},
3    name:
4      ['x', 'y', 'z', 'rx', 'ry', 'rz', 'use_nsp', 'nsp_type', 'nsp_p1', 'nsp_p2', 'nsp_p3', 'nsp_angle'],
5    position: [266.0, -306.6, 222.2, 88.0, 4.0, 75.0, 1, 1, 0.6765, 0.7364, 0.0, 15.0],
6    velocity: [],
7    effort: []
8  }" --once
9  publisher: beginning loop
10 publishing #1:
11 sensor_msgs.msg.JointState(header=std_msgs.msg.Header(stamp=builtin_interfaces.Time(sec=0, nanosec=0), frame_id='base_link'), name=['x', 'y', 'z', 'rx',
```

```
'ry', 'rz', 'use_nsp', 'nsp_type', 'nsp_p1', 'nsp_p2', 'nsp_p3', 'nsp_angle'],  
position=[266.0, -306.6, 222.2, 88.0, 4.0, 75.0, 1.0, 1.0, 0.6765, 0.7364,  
0.0, 15.0], velocity=[], effort=[])
```

3.1.3 天机机械臂逆解（NSP 臂角优化法）说明

IK + NSP 臂角优化（nsp_type=1, NEAR_DIR 近方向模式）：

nsp_type=1 表示"近方向"模式，需要提供一个 3D 方向向量作为臂角坐标引导。

典型工作流程：

- 先将机械臂运行/拖动到一个满意的参考构型
- 对该构型调用 FK+NSP（`compute_nsp: true`）获取 3×3 零空间投影矩阵
- 取 NSP 矩阵的**第一列**（即 `nsp_matrix[0]`，`nsp_matrix[3]`，`nsp_matrix[6]`，行优先）作为 `nsp_params`
- 后续所有逆解均以此方向引导，保证臂型一致性

例如，假设 FK+NSP 返回的 `nsp_matrix` 为：

Code block

```
1 [ 0.1234, 0.5678, 0.8012,  
2   0.3456, 0.7890, 0.2345,  
3   0.6789, 0.1234, 0.5678 ]
```

则 `nsp_params` 取第一列 = `[0.1234, 0.3456, 0.6789]`。

Code block

```
1 # 步骤 1: 先对参考构型做 FK+NSP, 获取 nsp_matrix  
2 ros2 service call /kinematics/forward_kinematics fl_msgs/srv/ForwardKinematics  
3   "{target_arm: 0, joint_positions: [21.8, -41.0, -4.74, -63.67, 10.15, 14.72,  
4   7.68], compute_nsp: true}"  
5  
6 # 从返回的 nsp_matrix 中提取第一列作为 nsp_params  
7  
8 # 步骤 2: 用 nsp_type=1 + 提取的方向向量做逆解
```

```
7  ros2 service call /kinematics/inverse_kinematics fl_msgs/srv/InverseKinematics
  \
8  "{
9    target_arm: 0,
10   target_xyzabc: [442.37, -63.45, 177.63, -92.10, 6.26, -131.52],
11   ref_joints: [21.8, -41.0, -4.74, -63.67, 10.15, 14.72, 7.68],
12   use_nsp: true,
13   nsp_type: 1,
14   nsp_params: [0.1234, 0.3456, 0.6789],
15   nsp_angle: 15.0
16  }"
```

参数说明：

- ‘ref_joints’：以机械臂当前位置作为参考位置
- ‘nsp_type=1’：逆解结果接近指定的臂角方向
- ‘nsp_params’：NSP 矩阵第一列（3D 方向向量）
- ‘nsp_angle’：在方向引导基础上调整臂角

3.2.1 关节状态获取（手臂）

Topic	/hal/joint_states
消息类型	sensor_msgs::msg::JointState
接入方式	发布（系统→具身）
接口说明	获取关节状态信息
接口参数	header：标准消息头，包含时间戳（stamp）、坐标系（frame_id）等元信息 string[] name：关节名称列表 double64[] position：每个关节的当前位置，获取机器人关节的旋转角度 double64[] velocity： double64[] effort：

关节状态信息，

```
1  user@user:~/zhou$ ros2 topic echo /hal/joint_states --once
2  header:
3  stamp:
```

```
4      sec: 1773821192
5      nanosec: 323696490
6      frame_id: base_link
7  name:
8  - arm_l_j1
9  - arm_l_j2
10 - arm_l_j3
11 - arm_l_j4
12 - arm_l_j5
13 - arm_l_j6
14 - arm_l_j7
15 - arm_r_j1
16 - arm_r_j2
17 - arm_r_j3
18 - arm_r_j4
19 - arm_r_j5
20 - arm_r_j6
21 - arm_r_j7
22 position:
23 - 89.9773178100586
24 - -90.01775360107422
25 - -89.99929809570312
26 - 4.806518700206652e-05
27 - 0.0043121338821947575
28 - 0.006900787353515625
29 - 0.0007690429920330644
30 - -89.99999237060547
31 - -90.0
32 - 89.99999237060547
33 - 0.0
34 - 0.0
35 - -6.86645489622606e-06
36 - -2.746581958490424e-05
37 velocity:
38 - 0.6197107325997138
39 - -0.7413407039717738
40 - 0.25967111379786023
41 - -0.5341213352784733
42 - -0.8441500175809448
43 - -0.4227027816351968
44 - 0.9843149369750697
45 - -0.7852999880361482
46 - 0.9801754651285723
47 - -0.5624276584202115
48 - 0.5018036337282634
49 - -0.06450587179977207
50 - -0.9769715757108548
```

```
51 - -0.5457854282755608
52 effort:
53 - 4.211410051344677
54 - 3.3402199323142185
55 - 2.3614247144329488
56 - 4.122592584540854
57 - 4.989063237225312
58 - 2.746372026274384
59 - 1.9561286625121057
60 - 3.45679096264335
61 - 3.6667642786643713
62 - 1.4791292221194097
63 - 4.326263988675867
64 - 3.701463701879106
65 - 1.8154149859391016
66 - 2.6151102566863305
67 ---
```

3.2.2 手臂末端位姿（未设置工具）

左臂

Topic	/state/left_arm/tcp_pos
消息类型	sensor_msgs::msg::JointState
接入方式	发布（系统→具身）
接口说明	获取左臂末端信息
接口参数	header：标准消息头，包含时间戳（stamp）、坐标系（frame_id）等元信息 string[] name：- x - y - z - rx - ry - rz double64[] position：空间坐标（x y z），及欧拉角（rx ry rz） double64[] velocity： double64[] effort：

Code block

```
1 ros2 topic echo /state/left_arm/tcp_pos --once
2 header:
3   stamp:
4     sec: 1781769442
```

```
5      nanosec: 276020943
6      frame_id: left_arm_tcp_pose
7      name:
8      - x
9      - y
10     - z
11     - rx
12     - ry
13     - rz
14     position:
15     - 327.702078567944
16     - 304.39732965220554
17     - 177.85634867897704
18     - -90.11090584416642
19     - 0.8298751327236195
20     - -88.40947570526943
21     velocity: []
22     effort: []
23     ---
```

右臂

Topic	/state/right_arm/tcp_pos
消息类型	sensor_msgs::msg::JointState
接入方式	发布（系统→具身）
接口说明	获取右臂末端信息
接口参数	header: 标准消息头，包含时间戳（stamp）、坐标系（frame_id）等元信息 string[] name: - x - y - z - rx - ry - rz double64[] position: 空间坐标（x y z），及欧拉角（rx ry rz） double64[] velocity: double64[] effort:

关节状态信息，

Code block

```
1  ros2 topic echo /state/right_arm/tcp_pos --once
2  header:
```

```
3  stamp:
4    sec: 1781769476
5    nanosec: 536010156
6  frame_id: right_arm_tcp_pose
7  name:
8    - x
9    - y
10   - z
11   - rx
12   - ry
13   - rz
14  position:
15    - 327.42603267490745
16    - -305.3489572400818
17    - 180.12690869455963
18    - 90.12606807731837
19    - 1.117242111841135
20    - 88.10985266871278
21  velocity: []
22  effort: []
23  ---
```

3.2.3 手臂各关节扰动力

左臂

Topic	/state/left_arm/joint_force
消息类型	sensor_msgs::msg::JointState
接入方式	发布（系统→具身）
接口说明	获取左臂关节扰动力信息
接口参数	header：标准消息头，包含时间戳（stamp）、坐标系（frame_id）等元信息 string[] name：- arm_l_j1 - arm_l_j2 - arm_l_j3 - arm_l_j4 - arm_l_j5 - arm_l_j6 - arm_l_j7 double64[] position： double64[] velocity： double64[] effort：关节扰动力，单位N·m

Code block

```
1 header:
2   stamp:
3     sec: 1783404233
4     nanosec: 565300824
5   frame_id: left_arm
6 name:
7 - arm_l_j1
8 - arm_l_j2
9 - arm_l_j3
10 - arm_l_j4
11 - arm_l_j5
12 - arm_l_j6
13 - arm_l_j7
14 position: []
15 velocity: []
16 effort:
17 - -2.8975491523742676
18 - -0.002732743276283145
19 - -0.061131205409765244
20 - -2.811760187149048
21 - -0.07295850664377213
22 - -1.844452977180481
23 - 0.05641617998480797
24 ---
```

右臂

Topic	/state/right_arm/joint_force
消息类型	sensor_msgs::msg::JointState
接入方式	发布（系统→具身）
接口说明	获取右臂关节扰动力信息
接口参数	header：标准消息头，包含时间戳（stamp）、坐标系（frame_id）等元信息 string[] name： - arm_r_j1 - arm_r_j2 - arm_r_j3 - arm_r_j4 - arm_r_j5 - arm_r_j6 - arm_r_j7 double64[] position： double64[] velocity： double64[] effort：关节扰动力，单位N·m

```
1  ros2 topic echo /state/right_arm/joint_force --once
2  header:
3    stamp:
4      sec: 1783404275
5      nanosec: 765324908
6    frame_id: right_arm
7  name:
8    - arm_r_j1
9    - arm_r_j2
10   - arm_r_j3
11   - arm_r_j4
12   - arm_r_j5
13   - arm_r_j6
14   - arm_r_j7
15  position: []
16  velocity: []
17  effort:
18    - 2.349184274673462
19    - 0.6220661401748657
20    - 0.4882524609565735
21    - -2.678506374359131
22    - -0.007693796884268522
23    - -0.3045937418937683
24    - -0.02564374916255474
25  ---
```

3.3 头部及躯干关节控制（弃用，使用第5部分接口）

~~注意：控制头部及躯干的关节时，必须通过插值算法，让对应关节每次只变化很小的幅度。~~

~~测试参考数据：1000hz的控制频率，每次变化0.001度。~~

Topic	/motion_ctl/joint_ctl
消息类型	sensor_msgs::msg::JointState
接入方式	订阅（具身→系统）
接口说明	控制关节运动
接口参数	header: 标准消息头，包含时间戳（stamp）、坐标系（frame_id）等元信息 string[] name: 关节名称列表 double64[] position: 指定每个关节的目标位置，控制机器人关节的旋转角度

```
double64[] velocity:-
```

```
double64[] effort:-
```

头部三电机转至零位

```
1  ros2 topic pub /motion_ctl/joint_ctl sensor_msgs/msg/JointState "{  
2  header: {stamp: {sec: 0, nanosec: 0}, frame_id: 'base_link'},  
3  name: ['head_j1', 'head_j2', 'head_j3'],  
4  position: [0.0, 0.0, 0.0],  
5  velocity: [],  
6  effort: []  
7  }" --once
```

腿部站立 (body_j4 body_j5两个电机角度只能为非正数!!)-

```
1  ros2 topic pub /motion_ctl/joint_ctl sensor_msgs/msg/JointState "{  
2  header: {stamp: {sec: 0, nanosec: 0}, frame_id: 'base_link'},  
3  name: ['body_j3', 'body_j4', 'body_j5'],  
4  position: [ 11.0, 25.0, 15.0],  
5  velocity: [],  
6  effort: []  
7  }" --once
```

3.4 头部及躯干关节状态获取

```
1  ros2 topic echo /hal/joint_states --once  
2  header:  
3    stamp:  
4      sec: 1776756184  
5      nanosec: 519518118  
6    frame_id: base_link  
7  name:  
8  - head_j1  
9  - head_j2  
10 - head_j3  
11 - body_j1
```

```

12 - body_j2
13 - body_j3
14 - body_j4
15 - body_j5
16 position:
17 - 0.0
18 - -5
19 - 0.0
20 - 0.0
21 - 0.0
22 - 0.0
23 - -6.00068664550781
24 - -5.00068664550781
25 velocity:
26 effort:
27 - 0.0
28 ---
29

```

3.5 灵巧手控制

参考“关节控制”

说明：

position字段表示手指弯曲的百分比，取值范围是[0.0, 100.0]，0.0是手指不弯曲，只需要变更关节名称：

```

1  关节定义：
2  hand_l_thumb      # 左手拇指弯曲
3  hand_l_index      # 左手食指弯曲
4  hand_l_middle     # 左手中指弯曲
5  hand_l_ring       # 左手无名指弯曲
6  hand_l_little     # 左手小指弯曲
7  hand_l_thumb_rot  # 左手拇指摆动
8
9  hand_r_thumb      # 右手拇指弯曲
10 hand_r_index      # 右手食指弯曲
11 hand_r_middle     # 右手中指弯曲
12 hand_r_ring       # 右手无名指弯曲
13 hand_r_little     # 右手小指弯曲
14 hand_r_thumb_rot  # 右手拇指摆动

```

测试:

```
1 ros2 topic pub /motion_ctl/joint_ctl sensor_msgs/msg/JointState "{
2   header: {stamp: {sec: 0, nanosec: 0}, frame_id: 'base_link'},
3   name: [
4     'hand_l_thumb', 'hand_l_index', 'hand_l_middle', 'hand_l_ring',
5     'hand_l_little', 'hand_l_thumb_rot',
6     'hand_r_thumb', 'hand_r_index', 'hand_r_middle', 'hand_r_ring',
7     'hand_r_little', 'hand_r_thumb_rot'
8   ],
9   position: [
10    0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
11    0.0, 0.0, 0.0, 0.0, 0.0, 0.0
12  ],
13  velocity: [],
14  effort: []
15 }" --once
```

右手握拳

```
1 ros2 topic pub /motion_ctl/joint_ctl sensor_msgs/msg/JointState "{
2   header: {stamp: {sec: 0, nanosec: 0}, frame_id: 'base_link'},
3   name: ['hand_r_thumb', 'hand_r_index', 'hand_r_middle', 'hand_r_ring',
4     'hand_r_little', 'hand_r_thumb_rot'],
5   position: [80.0, 90.0, 90.0, 90.0, 90.0, 30.0],
6   velocity: [],
7   effort: []
8 }" --once
```

右手张开

```
1 ros2 topic pub /motion_ctl/joint_ctl sensor_msgs/msg/JointState "{
2   header: {stamp: {sec: 0, nanosec: 0}, frame_id: 'base_link'},
3   name: ['hand_r_thumb', 'hand_r_index', 'hand_r_middle', 'hand_r_ring',
4     'hand_r_little', 'hand_r_thumb_rot'],
5   position: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
6   velocity: [],
7   effort: []
8 }" --once
```

左手张开

```
1 ros2 topic pub /motion_ctl/joint_ctl sensor_msgs/msg/JointState "{
2   header: {stamp: {sec: 0, nanosec: 0}, frame_id: 'base_link'},
3   name: ['hand_l_thumb', 'hand_l_index', 'hand_l_middle', 'hand_l_ring',
4   'hand_l_little', 'hand_l_thumb_rot'],
5   position: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
6   velocity: [],
7   effort: []
8 }" --once
```

左手握拳

```
1 ros2 topic pub /motion_ctl/joint_ctl sensor_msgs/msg/JointState "{
2   header: {stamp: {sec: 0, nanosec: 0}, frame_id: 'base_link'},
3   name: ['hand_l_thumb', 'hand_l_index', 'hand_l_middle', 'hand_l_ring',
4   'hand_l_little', 'hand_l_thumb_rot'],
5   position: [80.0, 90.0, 90.0, 90.0, 90.0, 30.0],
6   velocity: [],
7   effort: []
8 }" --once
```

3.6 灵巧手角度获取

参考“关节角度获取”

左手握紧，右手松开

```
1 ros2 topic echo /hal/joint_states --once
2 header:
3   stamp:
4     sec: 1775127499
5     nanosec: 330595738
6   frame_id: base_link
7 name:
8 - arm_l_j1
```

9 - arm_l_j2
10 - arm_l_j3
11 - arm_l_j4
12 - arm_l_j5
13 - arm_l_j6
14 - arm_l_j7
15 - arm_r_j1
16 - arm_r_j2
17 - arm_r_j3
18 - arm_r_j4
19 - arm_r_j5
20 - arm_r_j6
21 - arm_r_j7
22 - hand_l_thumb
23 - hand_l_index
24 - hand_l_middle
25 - hand_l_ring
26 - hand_l_little
27 - hand_l_thumb_rot
28 - hand_r_thumb
29 - hand_r_index
30 - hand_r_middle
31 - hand_r_ring
32 - hand_r_little
33 - hand_r_thumb_rot
34 position:
35 - 34.39162826538086
36 - -89.74494171142578
37 - -89.77814483642578
38 - -71.14582824707031
39 - -1.8737045526504517
40 - 6.533219337463379
41 - -0.8427749872207642
42 - -95.47848510742188
43 - -88.97775268554688
44 - 87.00138854980469
45 - -80.29446411132812
46 - -0.4100441038608551
47 - 7.011460781097412
48 - 0.11965484917163849
49 - 80.0
50 - 90.0
51 - 90.0
52 - 90.0
53 - 90.0
54 - 30.0
55 - 0.0

56	- 0.0
57	- 0.0
58	- 0.0
59	- 0.0
60	- 0.0
61	velocity:
62	- -0.71101485865144
63	- -0.18713500037113617
64	- -0.3159590910743668
65	- -0.7488720157266211
66	- -0.2875533688119726
67	- -0.014235112270153505
68	- -0.37175644863662327
69	- 0.14383366611578063
70	- -0.2934707711445156
71	- 0.5820058177830947
72	- -0.4846675580041545
73	- -0.6614824928308352
74	- -0.8914386824472424
75	- -0.4214041750553229
76	- 0.0
77	- 0.0
78	- 0.0
79	- 0.0
80	- 0.0
81	- 0.0
82	- 0.0
83	- 0.0
84	- 0.0
85	- 0.0
86	- 0.0
87	- 0.0
88	effort:
89	- 0.9491601622629837
90	- 2.5982051273578177
91	- 3.2308024141513525
92	- 2.566717971233014
93	- 1.6466575544854287
94	- 3.3852739867315367
95	- 0.6080047275187501
96	- 1.5209888121093988
97	- 2.005607229410045
98	- 2.8317062789157745
99	- 4.065696699725653
100	- 3.684634582913557
101	- 2.2221517424479207
102	- 0.6358486094628575

103	- 0.0
104	- 0.0
105	- 0.0
106	- 0.0
107	- 0.0
108	- 0.0
109	- 0.0
110	- 0.0
111	- 0.0
112	- 0.0
113	- 0.0
114	- 0.0
115	---
116	

3.7 夹爪控制

左夹爪控制

Topic	/motion_ctl/gripper/left
消息类型	control_msgs::msg::GripperCommand
接入方式	订阅（具身→系统）
接口说明	控制左夹爪运动
接口参数	float64 position：角度，100.0为闭合，0.0为完全张开 float64 max_effort：

右夹爪控制

Topic	/motion_ctl/gripper/right
消息类型	control_msgs::msg::GripperCommand
接入方式	订阅（具身→系统）
接口说明	控制右夹爪运动
接口参数	float64 position：角度，100.0为闭合，0.0为完全张开 float64 max_effort：可选

左夹爪闭合

```
1 ros2 topic pub /motion_ctl/gripper/left control_msgs/msg/GripperCommand "{position: 100.0, max_effort: 10.0}" --once
```

右夹爪张开

```
1 ros2 topic pub /motion_ctl/gripper/right control_msgs/msg/GripperCommand "{position: 0.0, max_effort: 10.0}" --once
```

can重启

```
1 已配置can开机自启
2 夹爪不可控时，需要确认can状态
3 user@ms-thor:~$ ip link | grep can
4 2: can0: <NO-CARRIER,NOARP,UP,ECHO> mtu 72 qdisc pfifo_fast state DOWN mode
  DEFAULT group default qlen 10
5     link/can
6 3: can1: <NO-CARRIER,NOARP,UP,ECHO> mtu 72 qdisc pfifo_fast state DOWN mode
  DEFAULT group default qlen 10
7     link/can
8 4: can2: <NOARP,ECHO> mtu 16 qdisc noop state DOWN mode DEFAULT group default
  qlen 10
9     link/can
10 5: can3: <NOARP,ECHO> mtu 16 qdisc noop state DOWN mode DEFAULT group default
  qlen 10
11    link/can
12    状态为DOWN，夹爪无法控制！！
13
14 需要手动启动can:
15 sudo ip link set can0 down
16 sudo ip link set can0 type can bitrate 1000000 dbitrate 5000000 fd on
17 sudo ip link set up can0
18 sudo ip link set can1 down
19 sudo ip link set can1 type can bitrate 1000000 dbitrate 5000000 fd on
20 sudo ip link set up can1
```

3.8 夹爪角度获取

左夹爪角度获取

Topic	/motion_ctl/gripper/left/state
消息类型	sensor_msgs::msg::JointState
接入方式	发布（系统→具身）
接口说明	获取左夹爪状态
接口参数	header: 标准消息头，包含时间戳（stamp）、坐标系（frame_id）等元信息 string[] name: left_gripper_position double64[] position: 左夹爪角度，100.0为闭合，0.0为完全张开

右夹爪角度获取

Topic	/motion_ctl/gripper/right/state
消息类型	sensor_msgs::msg::JointState
接入方式	发布（系统→具身）
接口说明	获取右夹爪状态
接口参数	float64 position: 角度，100.0为闭合，0.0为完全张开header: 标准消息头，包含时间戳（stamp）、坐标系（frame_id）等元信息 string[] name: right_gripper_position double64[] position: 右夹爪角度，100.0为闭合，0.0为完全张开

实测数据

```
1  ros2 topic echo /motion_ctl/gripper/left/state --once
2  header:
3    stamp:
4      sec: 1781234224
5      nanosec: 10994965
6    frame_id: left_gripper_state
7  name:
8    - left_gripper_position
9  position:
10   - 0.0
11 velocity: []
12 effort: []
13 ---
```

3.9 Camera

Code block

- 1 压缩格式图片的相关topic名称:
- 2 左手腕:
- 3 名称: /camera/left_wrist/color/image_raw/compressed
- 4 类型: sensor_msgs/msg/CompressedImage
- 5
- 6 右手腕:
- 7 名称: /camera/right_wrist/color/image_raw/compressed
- 8 类型: sensor_msgs/msg/CompressedImage
- 9
- 10 头部:
- 11 名称: /camera/head/color/image_raw/compressed
- 12 类型: sensor_msgs/msg/CompressedImage

头部

发布者, 系统→具身

查看camera相关topic的消息类型

- 1 `ros2 topic type /camera/head/color/image_raw`
- 2 `sensor_msgs/msg/Image`
- 3
- 4 `ros2 topic type /camera/right_wrist/color/image_raw`
- 5 `sensor_msgs/msg/Image`
- 6
- 7 `ros2 topic type /camera/left_wrist/color/image_raw`
- 8 `sensor_msgs/msg/Image`

头部相关topic:

- 1 `ros2 topic list`
- 2 `/camera/head/accel/imu_info`
- 3 `/camera/head/accel/metadata`
- 4 `/camera/head/accel/sample`
- 5 `/camera/head/color/camera_info`
- 6 `/camera/head/color/image_raw`
- 7 `/camera/head/color/metadata`
- 8 `/camera/head/depth/camera_info`
- 9 `/camera/head/depth/image_rect_raw`

```
10 /camera/head/depth/metadata
11 /camera/head/extrinsics/depth_to_accel
12 /camera/head/extrinsics/depth_to_color
13 /camera/head/extrinsics/depth_to_depth
14 /camera/head/extrinsics/depth_to_gyro
15 /camera/head/extrinsics/depth_to_infra1
16 /camera/head/gyro/imu_info
17 /camera/head/gyro/metadata
18 /camera/head/gyro/sample
19 /camera/head/infra1/camera_info
20 /camera/head/infra1/image_rect_raw
21 /camera/head/infra1/metadata
```

手腕部

发布者，系统→具身

以下示例为右手腕，左手腕将right替换为left。

部署方式为多进程部署，每个相机单独启动一个ros node。

相关topic（右手腕）

```
1 ros2 topic list
2 /camera/right_wrist/color/camera_info
3 /camera/right_wrist/color/image_raw
4 /camera/right_wrist/color/metadata
5 /camera/right_wrist/depth/camera_info
6 /camera/right_wrist/depth/image_rect_raw
7 /camera/right_wrist/depth/metadata
8 /camera/right_wrist/extrinsics/depth_to_color
9 /camera/right_wrist/extrinsics/depth_to_depth
10 /camera/right_wrist/extrinsics/depth_to_infra1
11 /camera/right_wrist/infra1/camera_info
12 /camera/right_wrist/infra1/image_rect_raw
13 /camera/right_wrist/infra1/metadata
```

左手腕的相关topic

```
1 /camera/left_wrist/color/camera_info
2 /camera/left_wrist/color/image_raw
3 /camera/left_wrist/color/metadata
4 /camera/left_wrist/depth/camera_info
```

```
5 /camera/left_wrist/depth/image_rect_raw
6 /camera/left_wrist/depth/metadata
7 /camera/left_wrist/extrinsics/depth_to_color
8 /camera/left_wrist/extrinsics/depth_to_depth
9 /camera/left_wrist/extrinsics/depth_to_infra1
10 /camera/left_wrist/infra1/camera_info
11 /camera/left_wrist/infra1/image_rect_raw
12 /camera/left_wrist/infra1/metadata
```

深度图相关：

Code block

```
1 头部
2 /camera/head/depth/image_raw
3 /camera/head/depth/image_raw/compressedDepth
4
5 右手腕
6 /camera/right_wrist/depth/image_rect_raw
7 /camera/right_wrist/depth/image_rect_raw/compressedDepth
8
9 左手腕
10 /camera/left_wrist/depth/image_rect_raw
11 /camera/left_wrist/depth/image_rect_raw/compressedDepth
```

3.10 底盘

具体使用，参考供应商接口文档说明

```
1 user@ms-thor:~/zhou$ ros2 topic list
2 /chassis_info
3 /cloudRegistered
4 /controller/odom
5 /driver/battery/Data
6 /driver/camera/point_cloud/Data
7 /driver/imu/Data
8 /driver/imu/Data2
9 /driver/imu/RawData
10 /driver/io/Data
11 /driver/lidar/lidar_back/imu/Data
12 /driver/lidar/lidar_back/point_cloud/Data
```

```
13 /driver/lidar/lidar_front/imu/Data
14 /driver/lidar/lidar_front/point_cloud/Data
15 /driver/lidar/point_cloud/Data
16 /driver/motor/Data
17 /driver/motor/SetMotorCmd
18 /driver/radar/Data
19 /driver/shaft/motor/SetMotorCmd
20 /driver/stm32/Data
21 /gamepad_data
22 /global_map
23 /hal/joint_states
24 /imu_base
25 /initialpose
26 /location_info
27 /loop_src
28 /loop_tar
29 /map_cmap
30 /map_init
31 /map_path
32 /map_pmap
33 /map_scan
34 /map_true
35 /motion_ctrl/gripper
36 /move/AutoMoveCmd
37 /move/ManualMoveCmd
38 /move/State
39 /odomAftUpdate
40 /parameter_events
41 /robo/manipulation/move_to_pose_cmd
42 /rosout
43 /slam/LocationData
44 /steward/alarm/Data
45 /tf
46 /tts_say
```

3.11 升降装置

抬升

Code block

```
1 ros2 topic pub --once /control/waist/updown std_msgs/msg/Bool "{data: true}"
```

下降

```
1 ros2 topic pub --once /control/waist/updown std_msgs/msg/Bool "{data: false}"
```

配置: waist_updown_config.ini

Code block

```
1 # Default motion settings for waist_updown_ros2.py.
2 # ROS 2 command-line parameters can still override these values at startup.
3
4 [motion]
5 distance_m = 0.05
6 duration_s = 3.0
```

3.12 遥控控制位姿

进入位姿1:

Code block

```
1 ros2 topic pub /motion_ctl/whole_ctl std_msgs/msg/Int32 "{data: 1}" --once
```

进入位姿2:

Code block

```
1 ros2 topic pub /motion_ctl/whole_ctl std_msgs/msg/Int32 "{data: 2}" --once
```

配置文件: whole_body_config.ini

Code block

```
1 # =====
2 # 全身一键复位/位姿切换配置文件
3 # ros2 topic pub /motion_ctl/whole_ctl std_msgs/msg/Int32 "{data: 1}" --once
4 # =====
5
6 [motion]
7 # 插值步数 (越大越平滑)
8 interpolation_steps = 500
9 # 每段运动时长 (秒)
10 segment_duration_s = 3.0
11
```

```

12 # =====
13 # 姿态定义
14 # 说明:
15 #   - 只配置需要移动的关节, 未配置目标角度的部位保持当前位置
16 #   - 左臂必须7个关节全配, 右臂必须7个关节全配
17 # =====
18
19 # ----- 姿态 1: 低姿态 (装箱/折叠) -----
20 [pose_1]
21 # 左臂 (7个关节必须全配)
22 arm_l = 80.0, -90.0, -90.0, -70.0, 0.0, 0.0, 0.0
23 # 右臂 (7个关节必须全配)
24 arm_r = -80.0, -90.0, 90.0, -70.0, 0.0, 0.0, 0.0
25 # 头部
26 head = 0.0, 0.0, 0.0
27 # 躯干
28 body = 0.0, 0.0, -2.0, -6.0, -5.0
29 # 描述
30 description = 装箱模式
31
32 [pose_2]
33 # 左臂 (7个关节必须全配)
34 arm_l = 90.0, -90.0, -90.0, -120.0, 0.0, 0.0, 0.0
35 # 右臂 (7个关节必须全配)
36 arm_r = -90.0, -90.0, 90.0, -120.0, 0.0, 0.0, 0.0
37 # 头部
38 head = 15.0, -10.0, 10.0
39 # 躯干
40 body = 5.0, -5.0, -12.0, -18, -10.0
41 # 描述
42 description = 装箱模式

```

4. Ros2 Service列表

手臂拖动模式设置

service定义:

SetDragMode.srv

```

1 # 设置拖动模式
2
3 # 请求
4 uint8 ARM_LEFT = 1

```

```

5  uint8 ARM_RIGHT = 2
6  uint8 ARM_BOTH = 3
7
8  uint8 target_arm      # 目标手臂: 1-左臂, 2-右臂, 3-双臂
9  uint8 drag_type       # 0-退出拖动, 1-关节空间拖动, 2-笛卡尔X, 3-笛卡尔Y, 4-笛卡尔Z, 5-
    笛卡尔旋转
10 uint32 duration_ms    # 拖动持续时间(毫秒), 0表示一直持续直到手动退出
11 ---
12 # 响应
13 bool success
14 string message

```

测试:

进入拖动模式:

```

1  ros2 service call /motor_ctl/set_drag_mode f1_msgs/srv/SetDragMode "
    {target_arm: 1, drag_type: 1, duration_ms: 0}"
2  waiting for service to become available...
3  requester: making request: f1_msgs.srv.SetDragMode_Request(target_arm=1,
    drag_type=1, duration_ms=0)
4
5  response:
6  f1_msgs.srv.SetDragMode_Response(success=True, message='Drag mode set
    successfully')

```

退出拖动模式:

```

1  ros2 service call /motor_ctl/set_drag_mode f1_msgs/srv/SetDragMode "
    {target_arm: 1, drag_type: 0, duration_ms: 0}"
2  waiting for service to become available...
3  requester: making request: f1_msgs.srv.SetDragMode_Request(target_arm=1,
    drag_type=0, duration_ms=0)
4
5  response:
6  f1_msgs.srv.SetDragMode_Response(success=True, message='Drag mode set
    successfully')

```

手臂运动模式设置

SetControlMode.srv

```

1  # 设置机器人运动控制模式
2  # 左臂/右臂/双臂控制
3
4  # 请求
5  uint8 ARM_LEFT = 1
6  uint8 ARM_RIGHT = 2
7  uint8 ARM_BOTH = 3
8
9  uint8 target_arm # 目标手臂: 1-左臂, 2-右臂, 3-双臂
10 uint8 mode # 模式: 1-位置模式, 3-扭矩模式
11 ---
12 # 响应
13 bool success
14 string message

```

测试

```

1  ros2 service call /motor_ctl/set_control_mode f1_msgs/srv/SetControlMode "
    {target_arm: 1, mode: 3}"

```

手臂运动扭矩模式设置阻抗类型

SetImpedanceType.srv

消息定义

```

1  # 设置力矩模式下的阻抗类型
2  # 仅在扭矩模式下有效
3
4  # 请求
5  uint8 ARM_LEFT = 1
6  uint8 ARM_RIGHT = 2
7  uint8 ARM_BOTH = 3
8
9  uint8 target_arm # 目标手臂: 1-左臂, 2-右臂, 3-双臂
10 uint8 imp_type # 类型: 1-关节阻抗, 2-坐标阻抗, 3-力控
11 ---
12 # 响应
13 bool success
14 string message

```

测试:

```

1 # 设置左臂为关节阻抗 (type=1)
2 ros2 service call /motor_ctl/set_impedance_type f1_msgs/srv/SetImpedanceType "
  {target_arm: 1, imp_type: 1}"

```

手臂运动学正逆解

正解

srv定义: ForwardKinematics.srv

```

1 # 正运动学求解服务
2 # 输入关节角度, 输出末端位姿矩阵
3
4 # 手臂标识常量
5 uint8 ARM_LEFT = 0
6 uint8 ARM_RIGHT = 1
7
8 # 请求
9 uint8 target_arm # 目标手臂: 0-左臂(A), 1-右臂(B)
10 float64[] joint_positions # 7个关节角度(单位: 度)
11 bool compute_nsp # 是否同时计算零空间投影矩阵(NSP)
12 ---
13 # 响应
14 bool success
15 string message
16 float64[] pose_matrix # 4x4位姿矩阵, 行优先展开为16个元素
17 float64[] nsp_matrix # 3x3零空间投影矩阵, 行优先展开为9个元素
18 # 仅当 compute_nsp=true 时有效
19 float64[] xyzabc # 位姿信息XYZ及欧拉角ABC (单位: mm/度), 欧拉角顺序是
  XYZ

```

测试:

```

1 ros2 service call /kinematics/forward_kinematics
  f1_msgs/srv/ForwardKinematics "{target_arm: 0, joint_positions:
  [90,-90,-90,-100,0,0,0], compute_nsp: false}"
2 waiting for service to become available...
3 requester: making request:
  f1_msgs.srv.ForwardKinematics_Request(target_arm=0, joint_positions=[90.0,
  -90.0, -90.0, -100.0, 0.0, 0.0, 0.0], compute_nsp=False)
4
5 response:

```

```

6  fl_msgs.srv.ForwardKinematics_Response(success=True, message='FK computed
    successfully', pose_matrix=[-0.1736481775310885, 4.5988695296101047e-10,
    0.9848077530122071, 423.9120381775523, -0.9848077530122071,
    -4.5988695296101047e-10, -0.1736481775310885, 233.70443494400456,
    3.730417055331584e-10, -1.0, 5.327587680371682e-10, 174.50000008747398, 0.0,
    0.0, 0.0, 1.0], nsp_matrix=[], xyzabc=[423.9120381775523, 233.70443494400456,
    174.50000008747398, -89.99999996947517, -2.1373714946563052e-08,
    -99.99999999230032])

```

逆解

srv定义: InverseKinematics.srv

```

1  # 逆运动学求解服务
2  # 输入目标位姿矩阵和参考关节角度, 输出求解的关节角度
3  # 支持可选的零空间(NSP)优化
4
5  # 手臂标识常量
6  uint8 ARM_LEFT = 0
7  uint8 ARM_RIGHT = 1
8
9  # 请求
10 uint8 target_arm          # 目标手臂: 0-左臂(A), 1-右臂(B)
11 float64[] target_pose    # 目标4x4位姿矩阵, 行优先展开为16个元素
12 float64[] ref_joints     # 参考关节角度(度), 7个元素
13 float64[] target_xyzabc  # 目标位姿, XYZRxRyRz, 空间坐标及欧拉角
14
15 # 零空间优化参数 (可选)
16 bool use_nsp              # 是否使用零空间优化
17 uint8 nsp_type            # 零空间类型: 0-近参考(NEAR_REF), 1-近方向
    (NEAR_DIR)
18 float64[] nsp_params     # 零空间方向参数(3个元素), nsp_type=1时需要
19 float64 nsp_angle        # 零空间角度调整(度), 用于臂角优化
20 ---
21 # 响应
22 bool success
23 string message
24 float64[] joint_positions # 逆解关节角度(度), 7个元素
25 bool is_out_range        # 当前位姿是否超出位置可达空间
26 bool[] is_singular       # 各关节是否发生奇异(7个元素)
27 bool is_joint_exceeded   # 是否有关节超出位置正负限制
28 bool[] joint_exceeded_tags # 各关节是否超出位置限制(7个元素)
29 float64[] joint_limit_pos # 各关节正限制(度), 7个元素
30 float64[] joint_limit_neg # 各关节负限制(度), 7个元素
31 int32 result_num         # 逆解结果数量
32

```

测试:

```
1  ros2 service call /kinematics/inverse_kinematics
   f1_msgs/srv/InverseKinematics "{
2      target_arm: 0,
3      target_pose: [
4          0, 0, 0, 0,
5          0, 0, 0, 0,
6          0, 0, 0, 0,
7          0, 0, 0, 0
8      ],
9      ref_joints: [90, -90, -90, -90, 0, 0, 0],
10     target_xyzabc: [381,98,174.5,-89,0,-120],
11     use_nsp: false
12 }"
13 waiting for service to become available...
14 requester: making request:
   f1_msgs.srv.InverseKinematics_Request(target_arm=0, target_pose=[0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0],
ref_joints=[90.0, -90.0, -90.0, -90.0, 0.0, 0.0, 0.0], xyzabc=[381.0, 98.0,
174.5, -89.0, 0.0, -120.0], use_nsp=False, nsp_type=0, nsp_params=[],
nsp_angle=0.0)
15
16 response:
17 f1_msgs.srv.InverseKinematics_Response(success=True, message='IK computed
successfully', joint_positions=[90.02872950197087, -90.65289196618096,
-89.99967265743788, -120.04815235609311, 0.5654180553840433,
0.017813030644674654, 0.673367529339032], is_out_range=False, is_singular=
[False, False, False, False, False, False, False], is_joint_exceeded=False,
joint_exceeded_tags=[False, False, False, False, False, False, False],
joint_limit_pos=[170.0, 120.0, 170.0, 60.0, 170.0, 60.0, 90.0],
joint_limit_neg=[-170.0, -94.0, -170.0, -145.0, -170.0, -60.0, -90.0],
result_num=4)
```

Code block

```
1  ros2 service call /kinematics/inverse_kinematics f1_msgs/srv/InverseKinematics
   "{
2      target_arm: 0,
3      target_pose: [
4          0, 0, 0, 0,
```

```

5      0, 0, 0, 0,
6      0, 0, 0, 0,
7      0, 0, 0, 0
8  ],
9  ref_joints: [104.28, -93.47, -101.29, -107.66, 13.00, 2.28, -15.31],
10 target_xyzabc: [442.37, -63.45, 174.5, -92, 6.26, -131],
11 use_nsp: true,
12 nsp_type: 1,
13 nsp_params: [0.6765, -0.7364, 0.0],
14 nsp_angle : 15.0
15 }"

```

5. UDP与主站通信接口

使用范围：

- 替换3.3部分的ros控制接口，改为与主站程序直接通信
- 控制对象为8个电机：头部3个关节电机，躯干5个关节电机
- 获取电机返回角度的接口，从ros中读取，参考3.4部分

限位说明：

控制端程序必须添加限位！！

Code block

序号	电机名称	限位角度及方向说明
0	head_j1	-40° ~ +40°，yaw，摇头，正向为左摇头
1	head_j2	-25° ~ +25°，pitch，抬头，正向为上抬头
2	head_j3	-15° ~ +15°，roll，歪头，正向为左歪头
3	body_j1	-130° ~ +130°，腰部yaw，正向为向左
4	body_j2	-10° ~ +10°，腰部roll，正向为向左
5	body_j3	-70° ~ 0°，腰部pitch，只能俯身（方向为负），不可后仰，只能为负值，绝对值越大越前倾
6	body_j4	-55° ~ 0°，屈膝，绝对值越大姿态越高，只能为负值
7	body_j5	-30° ~ 0°，脚踝，绝对值越大姿态越高，只能为负值

调试说明：

接口调试阶段，必须小角度，慢速调试，做好急停操作准备！！

UDP通信接口：

控制参数：电机id、目标角度

参考以下接口：

① UDP协议，ip：127.0.0.1，端口：50001，seq默认传1

② 电机id与电机name映射关系：

Code block

1	0	head_j1
2	1	head_j2
3	2	head_j3
4	3	body_j1
5	4	body_j2
6	5	body_j3
7	6	body_j4
8	7	body_j5

③ 目标角度target_angle：单位度

④ 其他字段传0

Code block

```
1 def send_command(motor_id, target_angle, seq=1, host='127.0.0.1', port=50001):
2     """
3     发送UDP控制命令
4     """
5     fmt = '<I B f f f I'
6     velocity = 0.0
7     acceleration = 0.0
8     crc = 0
9
10    data = struct.pack(fmt, seq, motor_id, target_angle, velocity,
11                        acceleration, crc)
12
13    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
14    sock.sendto(data, (host, port))
15    sock.close()
16
17    print(f"[SEND] motor_id={motor_id}, angle={target_angle}°, seq={seq}")
18    print(f"      data={data.hex()}")
19
20 def send_multiple_commands(commands, host='127.0.0.1', port=50001,
21                             delay_ms=50):
22     for i, (motor_id, angle) in enumerate(commands):
23         send_command(motor_id, angle, seq=i+1, host=host, port=port)
```

```
22         if delay_ms > 0 and i < len(commands) - 1:
23             time.sleep(delay_ms / 1000.0)
```

版本号

v1.1	初版 20260601
v1.2	① 调整机械臂的限位角度 ② 头部及躯干部电机控制改为UDP，添加第5部分接口说明 20260609
v1.3	20260612 ① 添加双臂两个topic，发布末端位姿（x y z rx ry rz，未设置工具） ② 修改获取夹爪状态的topic的消息类型，添加时间戳
v1.4	20260618 ① 添加两个topic，通过末端位姿控制臂运动，章节3.1.2 ② 3.9camera部分，添加压缩图片格式的相关topic信息 ③ 正逆解service部分修改
v1.5	20260626 ① 修改3.1.2 位姿控制接口，新增NSP相关参数 ② 新增3.1.3 新增位姿NSP计算使用说明